

Timer HAL Implementation Using STM32 Nucleo-F103RB

STM32F103RB Microcontroller Hardware Abstraction Layer (HAL)

Prepared By: Ahsan Basharat | Status: Intern | Submitted To: Sir Yasir

Objectives

- To understand the basic working principle of the TIM2 timer in the STM32F103RB microcontroller.
- To configure TIM2 using the STM32CubeMX and STM32 HAL Library.
- To configure the timer with an appropriate prescaler and period for generating accurate time intervals.
- To design and implement a custom TIMER_HAL abstraction layer for timer control.
- To generate periodic timer ticks using the timer interrupt.
- To use the timer tick to control LED operations at specific time intervals.
- To gain practical experience in implementing non-blocking and reusable timer-based software for embedded applications.

Introduction

Timers are important peripherals in STM32 microcontrollers used for generating accurate delays, measuring time intervals, and executing tasks at regular intervals. In this project, a TIMER_HAL is developed for the STM32F103RB using the STM32 HAL library. It provides simple functions to initialize, start, stop, and monitor the timer. A software tick counter is updated through the timer interrupt to track elapsed time. This project demonstrates efficient and reliable timer management for embedded applications.

Timers

Timers are hardware peripherals in a microcontroller used to measure time, generate delays, and perform tasks at fixed time intervals. They work by counting clock pulses generated by the microcontroller. By configuring parameters such as the prescaler, counter, and period, a timer can generate precise time intervals.

In STM32 microcontrollers, timers can also be used for interrupts, PWM generation, event counting, and time measurement. They are widely used in applications where accurate and reliable timing is required.

Required Components

#	Component / Tool	Quantity	Description / Specifications
1	STM32F103RB	1	ARM Cortex-M3 (32-bit)
2	USB Cable / ST-Link V2	1	Hardware programmer and debugger
3	Breadboard	1	Standard solderless prototyping breadboard
4	Light Emitting Diode (LEDs)	4	5mm Red/Green LED indicator
5	Resistor (220 Ω)	1	Current-limiting resistor for LED circuit
6	Jumper Wires	Set	Male-to-Male and Male-to-Female wires
7	STM32CubeIDE/MX	Software	Integrated Development Environment for C/C++

Timer Working Principle

The STM32F103RB general-purpose timers consist of a 16-bit auto-reload counter driven by a programmable prescaler. The functional relationship between the input clock and overflow rate is governed by key register components:

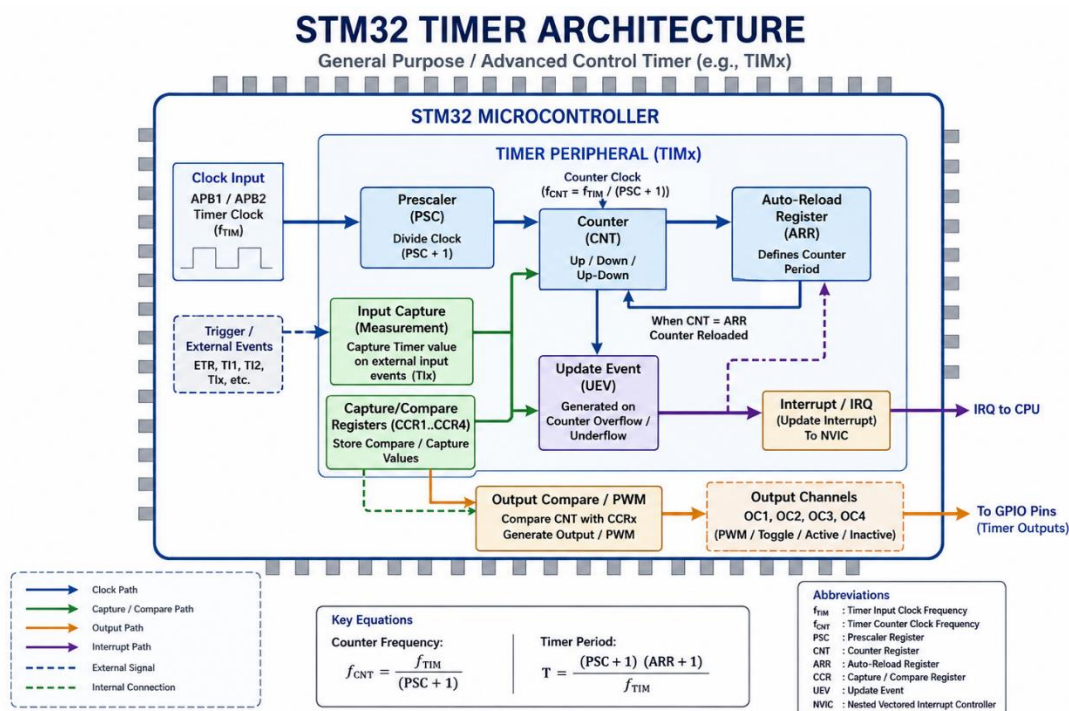
- Timer Clock (TIM): Input clock sourced from the APB1 or APB2 bus timer clock trees.
- Prescaler (PSC): Divides the peripheral clock by a factor between 1 and 65,536.
- Counter (CNT): 16-bit register incrementing on prescaled clock pulses.
- Auto-Reload Register (ARR): Holds maximum match value. Overflow occurs when CNT reaches ARR.
- Update Event (UEV) & Interrupt: Overflow triggers UEV, resets CNT, and issues an interrupt to the NVIC.

$$\text{Counter Frequency: } f_{CNT} = f_{TIM} / (PSC + 1)$$

$$\text{Timer Period: } T = ((PSC + 1) \times (ARR + 1)) / f_{TIM}$$

Timer Architecture

Timer architecture in an STM32 microcontroller consists of several hardware blocks that work together to measure time and generate precise timing events. The clock source provides the input clock to the timer, which is then divided by the prescaler to obtain the required timer frequency. The counter increments or decrements according to the selected counting mode. The Auto-Reload Register (ARR) defines the maximum counting value before the counter resets. Capture/Compare Registers (CCR) are used for input capture, output compare, and PWM generation. When a timer event occurs, it can generate an update event or interrupt, allowing the microcontroller to perform a specific task at precise time intervals.



Timer System Hierarchy

The STM32 timer system is organized into four main layers: Application, TIMER_HAL Wrapper, STM32 Driver Layer, and Hardware. The Application Layer contains functions such as LED sequencing, periodic tasks, event scheduling, and non-blocking delays. The TIMER_HAL provides a simple and reusable interface through functions such as `TIMER_Init()`, `TIMER_Start()`, `TIMER_Stop()`, `TIMER_GetTick()`, and `TIMER_IsElapsed()`. The STM32 HAL Driver Layer handles timer initialization, interrupt control, and communication with the hardware peripheral. At the bottom, the STM32F103RB timer hardware performs counting, auto-reload, capture/compare, update-event generation, and interrupt generation. This layered structure separates application logic from hardware details, making the system easier to understand, maintain, and reuse.

Custom TIMER_HAL Architecture

The custom `TIMER_HAL` abstraction layer decouples application logic from low-level STM32 HAL structures. Its core structure encapsulates the peripheral handle and software tick counter:

TIMER_HAL Structure Definition

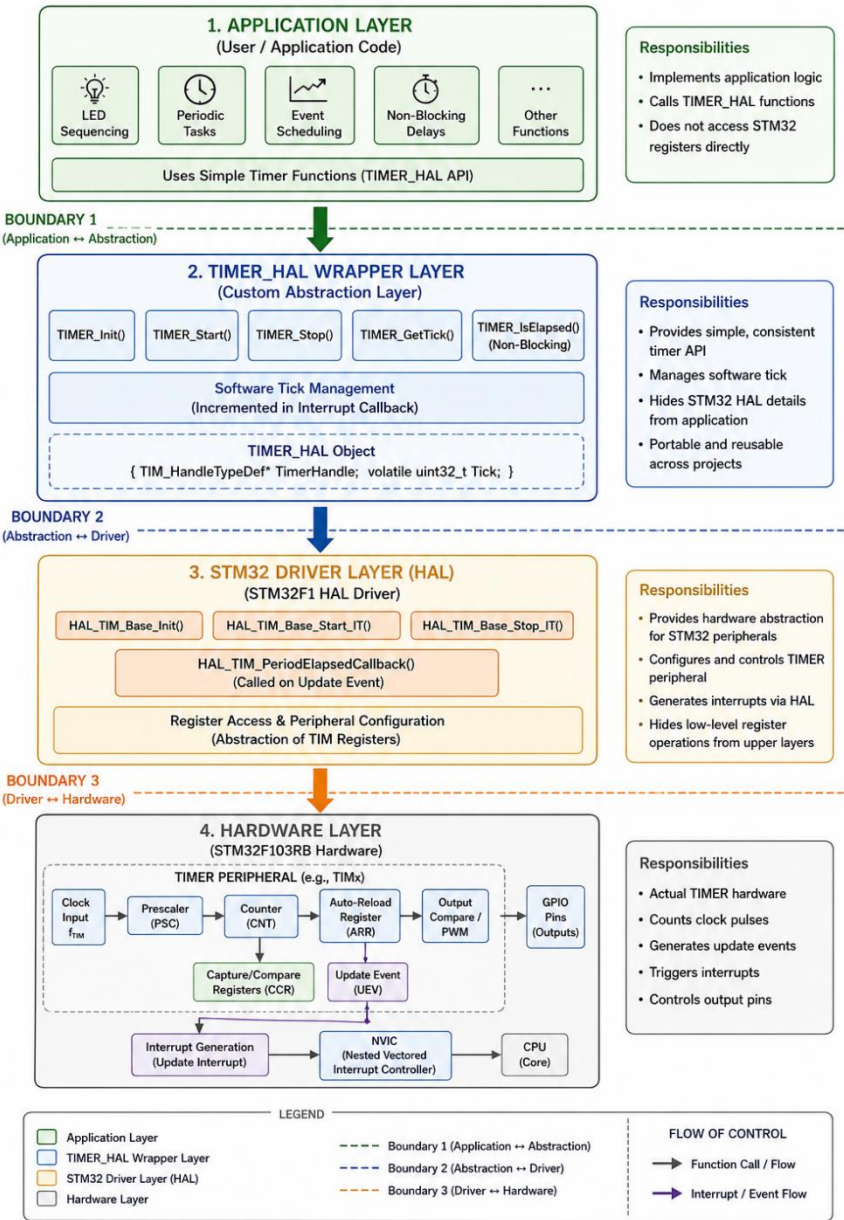
```
typedef struct
{
    TIM_HandleTypeDef *TimerHandle; /* Pointer to u
    volatile uint32_t Tick;          /* Software ti
} TIMER_HAL;
```

Component Roles:

- `TimerHandle`: Pointer to the vendor `TIM_HandleTypeDef` structure, binding the wrapper to hardware (e.g., `&tim2`).
- `Tick`: Unsigned 32-bit counter storing elapsed milliseconds. Marked volatile to prevent compiler register optimization during polling loops since it is modified asynchronously inside an ISR.

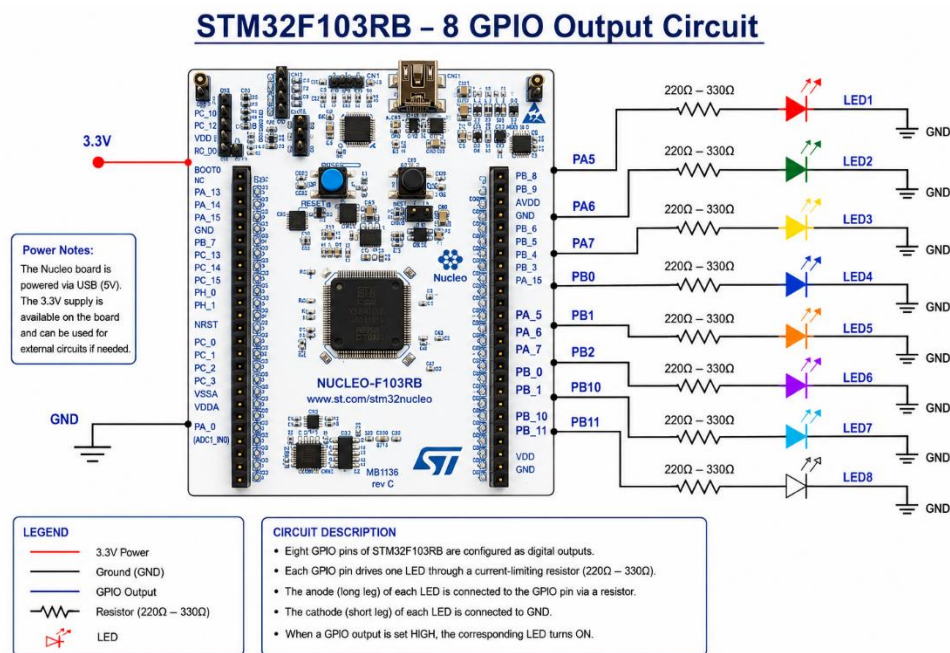
STM32 TIMER SYSTEM HIERARCHY

Layered Architecture and Boundaries



Circuit Description

- The STM32F103RB Nucleo board is configured with eight GPIO pins as digital outputs: PA5, PA6, PA7, PB0, PB1, PB2, PB10, and PB11.
- Each GPIO output is connected to one LED through a $220\ \Omega$ – $330\ \Omega$ current-limiting resistor.
- The other terminal of each LED is connected to GND, providing a complete current path.
- When a GPIO pin is driven HIGH, the corresponding LED turns ON; when it is LOW, the LED remains OFF.
- The resistors limit LED current and protect both the LEDs and STM32 GPIO pins from excessive current.
- This circuit demonstrates basic GPIO output control, allowing eight LEDs to be controlled independently for embedded-systems experiments.



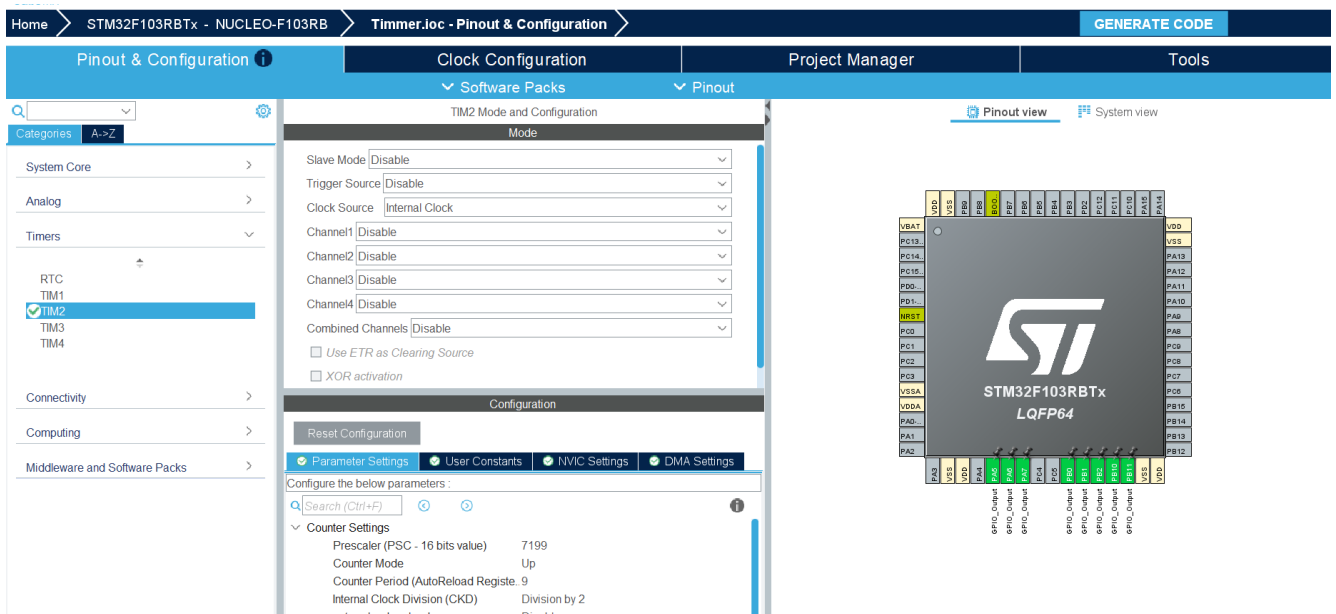
Procedure

1. Launch STM32CubeIDE and create a new project by selecting the target MCU STM32F103RB.
2. In the Device Configuration Tool (.ioc) Pinout view, configure PA5, PA6, PA7, PB0, PB1, PB2, PB10, and PB11 as GPIO_Output.
3. In the GPIO Configuration panel, set the output pins to the required GPIO output mode and default output level.
4. Assign User Labels to the pins, such as LED1, LED2, LED3, LED4, LED5, LED6, LED7, and LED8.
5. Save the IOC file to generate the required GPIO initialization code automatically.
6. Implement the program using HAL_GPIO_WritePin() or HAL_GPIO_TogglePin() to control the eight GPIO outputs.
7. Connect an LED with a suitable current-limiting resistor to each configured GPIO pin and connect the LED returns to GND.
8. Connect the ST-Link debugger to the PC and STM32F103RB board, build and flash the project, then test each LED to verify that all eight GPIO outputs operate correctly.

HAL Functions Used

Function Name	Purpose
TIMER_Init()	Initializes the Timer HAL object and sets its initial tick value and timer handle.
TIMER_Start()	Starts the STM32 timer in interrupt mode so that timer ticks are generated periodically.
TIMER_Stop()	Stops the timer and disables its periodic interrupt operation.
TIMER_GetTick()	Returns the current software timer tick value maintained by the timer interrupt.
TIMER_IsElapsed()	Checks whether the specified delay time has elapsed since a given starting tick. Returns 1U if elapsed and 0U otherwise.
TIMER_HAL	Stores the STM32 timer handle and the software tick counter used by the timer HAL.
TimerHandle	Pointer to the STM32 TIM_HandleTypeDef used to access the hardware timer.
Tick	Software counter that is incremented periodically inside the timer interrupt callback.

Code:



```
7 typedef struct
8 {
9     /**
10      * @brief Pointer to STM32 HAL timer handle.
11      */
12     TIM_HandleTypeDef *TimerHandle;
13 }
14 /**
15  * @brief Software timer tick counter.
16  * @details This variable is modified inside the timer interrupt
17  *          callback and therefore declared volatile.
18  */
19 volatile uint32_t Tick;
20 } TIMER_HAL;
21
22 /**
23  * @brief Initialize the timer HAL object.
24  * @param[in,out] timer Pointer to the timer HAL object.
25  */
26 void TIMER_Init(TIMER_HAL * const timer);
27
28 /**
29  * @brief Start the timer in interrupt mode.
30  * @param[in,out] timer Pointer to the timer HAL object.
31  */
32 void TIMER_Start(TIMER_HAL * const timer);
33
34 /**
35  * @brief Stop the timer interrupt.
36  * @param[in,out] timer Pointer to the timer HAL object.
37  */
38 void TIMER_Stop(TIMER_HAL * const timer);
39 /**
```

- TIMER_HAL structure stores the STM32 timer handle and software tick counter.
- TimerHandle points to the STM32 HAL timer peripheral.
- Tick keeps track of timer ticks and is updated inside the timer interrupt.
- TIMER_Init() initializes the timer HAL object.
- TIMER_Start() starts the timer in interrupt mode.
- TIMER_Stop() stops the timer interrupt.
- TIMER_GetTick() returns the current timer tick value.
- TIMER_IsElapsed() checks whether a specified delay has elapsed.

```

static TIMER_HAL *timerObject = NULL;

/*-----
 * Function : TIMER_Init()
 * Purpose  : Initialize timer object
 *-----*/
void TIMER_Init(TIMER_HAL * const timer)
{
    if (timer != NULL)
    {
        timer->Tick = 0U;

        timerObject = timer;
    }
}

/*-----*/
void TIMER_Start(TIMER_HAL * const timer)
{
    if (timer != NULL)
    {
        if (timer->TimerHandle != NULL)
        {
            (void)HAL_TIM_Base_Start_IT(timer->TimerHandle);
        }
    }
}

/*-----*/
void TIMER_Stop(TIMER_HAL * const timer)
{
    if (timer != NULL)
    {
        if (timer->TimerHandle != NULL)
        {
            (void)HAL_TIM_Base_Stop_IT(timer->TimerHandle);
        }
    }
}

uint32_t TIMER_GetTick(TIMER_HAL * const timer)
{
    uint32_t tick;

    tick = 0U;

    if (timer != NULL)
    {
        tick = timer->Tick;
    }

    return tick;
}

/*-----*/
 * Function : TIMER_IsElapsed()
 * Purpose  : Check whether delay has elapsed
 *-----*/
uint8_t TIMER_IsElapsed(TIMER_HAL * const timer,
                        uint32_t startTick,
                        uint32_t delayMs)
{
    uint32_t currentTick;
    uint32_t elapsedTime;
    uint8_t result;

    currentTick = 0U;
    elapsedTime = 0U;
    result = 0U;

    if (timer != NULL)
    {

```

Function: TIMER_Init()

Purpose: Initializes the Timer HAL object. Checks whether the timer pointer is not NULL. Sets the software Tick counter to 0U. Stores the timer object address in the global timerObject pointer. **timerObject** is used later by the timer interrupt callback to access the active timer. This function prepares the timer object before calling TIMER_Start().

TIMER_Start()

- **Purpose:** Starts the timer in **interrupt mode**.
- Checks that the timer pointer is not NULL.
- Checks that TimerHandle is valid.
- Calls **HAL_TIM_Base_Start_IT()** to start the hardware timer.
- The timer periodically generates **interrupts**.

TIMER_Stop()

- **Purpose:** Stops the timer and its interrupts.
- Checks that the timer pointer is not NULL.
- Checks that TimerHandle is valid.
- Calls **HAL_TIM_Base_Stop_IT()** to stop the timer.
- Prevents further timer interrupts from being generated.

TIMER_GetTick()

- **Purpose:** Gets the current software timer tick.
- Initializes the local tick variable to 0U.
- Checks whether the timer pointer is valid.
- Returns the value of **timer->Tick**.
- If the timer pointer is NULL, it safely returns **0U**.

TIMER_IsElapsed()

- **Purpose:** Checks whether the required delay has elapsed.
- Reads the current value of **timer->Tick**.
- Calculates elapsed time using:
elapsedTime = currentTick - startTick
- Compares the elapsed time with the requested **delayMs**.
- Returns **1U** when the delay has elapsed; otherwise, returns **0U**.
- The subtraction method also handles **timer tick overflow/wrap-around** correctly for unsigned integers.

Mian.c

```
#include "main.h"
#include "tim.h"
#include "gpio.h"
#include "GPIO_HAL.h"
#include "Timmer_HAL.h"

#define LED_DELAY_MS (100U)

void SystemClock_Config(void);

/* Timer object */
TIMER_HandleTypeDef Timer =
{
    &htim2,
    0U
};

/* LED PA5 */
GPIO_HandleTypeDef LED =
{
    GPIOA,
    GPIO_PIN_5
};

/* LED PA6 */
GPIO_HandleTypeDef LED1 =
{
    GPIOA,
    GPIO_PIN_6
};

/* Initialize timer */
TIMER_Init(&Timer);

/* Start timer */
TIMER_Start(&Timer);

/* LED array */
GPIO_HandleTypeDef *leds[8] =
{
    &LED,
    &LED1,
    &LED2,
    &LED3,
    &LED4,
    &LED5,
    &LED6,
    &LED7
};

/* All LEDs OFF */
for (i = 0U; i < 8U; i++)
{
    LED_Off(leds[i]);
}
```

```
/*=====*/
for (i = 0U; i < 8U; i++)
{
    /* Turn all LEDs OFF */
    for (j = 0U; j < 8U; j++)
    {
        LED_Off(leds[j]);
    }

    /* Turn current LED ON */
    LED_On(leds[i]);

    /* Wait */
    startTick = TIMER_GetTick(&Timer);
    while (TIMER_IsElapsed(&Timer,
                           startTick,
                           LED_DELAY_MS) == 0U)
    {
        /* Wait */
    }
}

/*=====*/
/* Reverse: LED6 -> LED1 */
/*=====*/
for (i = 6U; i > 0U; i--)
{
    /* Turn all LEDs OFF */
    for (j = 0U; j < 8U; j++)
    {
        LED_Off(leds[j]);
    }

    /* Turn current LED ON */
    LED_On(leds[i]);

    /* Wait */
    startTick = TIMER_GetTick(&Timer);
    while (TIMER_IsElapsed(&Timer,
                           startTick,
```

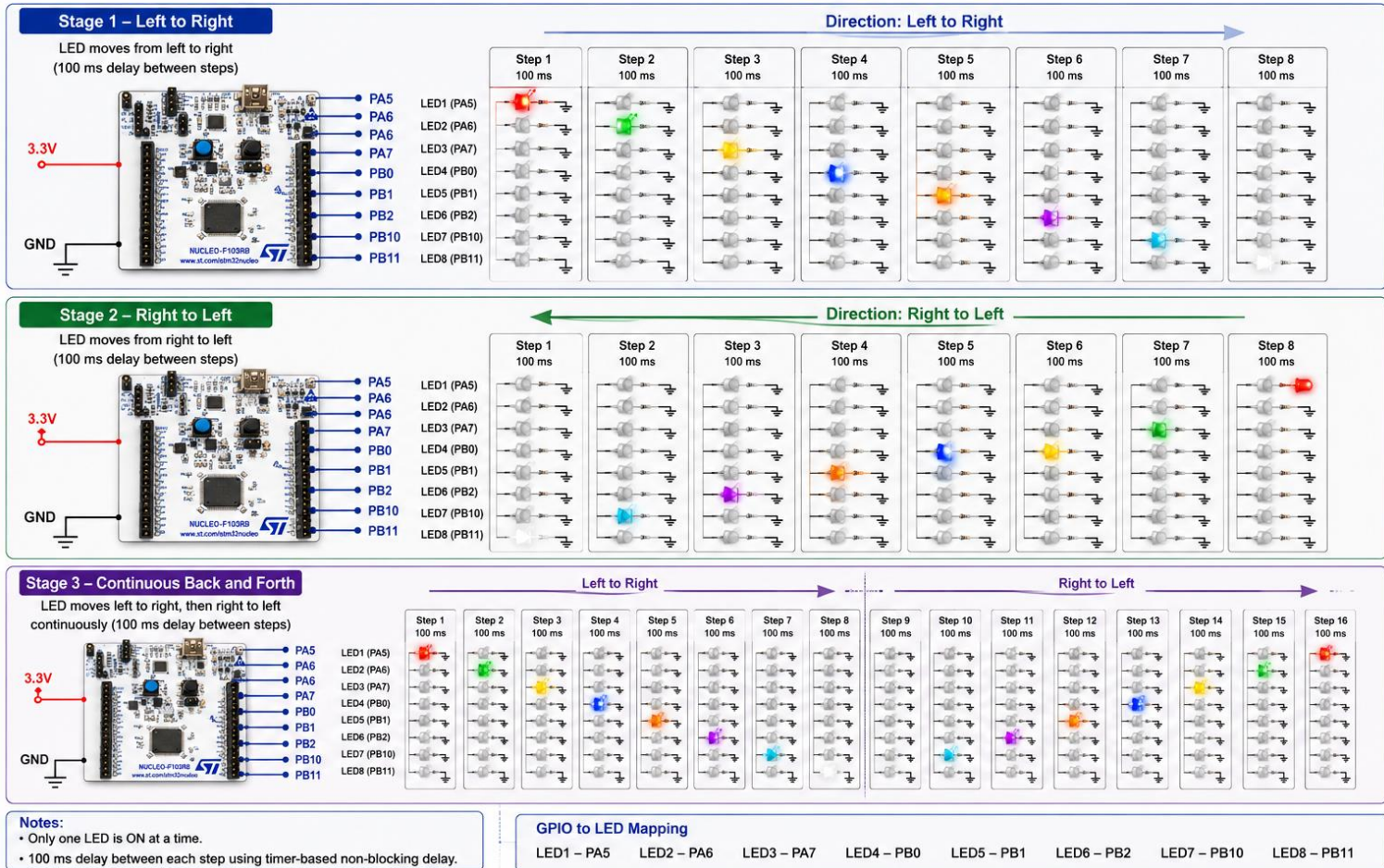
- LED_DELAY_MS (100U) sets the LED delay to 100 ms.
- SystemClock_Config() configures the STM32 system clock.
- TIMER_HAL Timer creates the timer object using TIM2.
- htim2 is the HAL timer handle connected to the timer object.
- GPIO_HAL LED configures PA5 as an LED output.
- GPIO_HAL LED1 configures PA6 as another LED output.
- These objects allow the program to control timer timing and LEDs through HAL wrappers.

- Initializes **8 LEDs** using LED_Init().
- Initializes the timer using **TIMER_Init()**.
- Starts the timer with **TIMER_Start()**.
- Creates an **array of 8 LED objects** for easy control.
- Uses a for loop to access all LEDs.
- Turns **all 8 LEDs OFF** using LED_Off().

- The first for loop moves the LED sequence **from left to right**.
- Before each step, **all 8 LEDs are turned OFF**.
- The current LED is then turned **ON**.
- TIMER_GetTick() records the starting time, and TIMER_IsElapsed() creates a **100 ms delay**.
- The second for loop moves the sequence **from right to left**.
- This creates a continuous **back-and-forth LED running effect**.

Program Output

STM32F103RB Timer-Based LED Running Light – Program Output



The STM32F103RB implements a timer-based running-light sequence using eight GPIO-controlled LEDs. The LEDs are connected to PA5, PA6, PA7, PB0, PB1, PB2, PB10, and PB11, with one LED activated at a time. A 100 ms timer delay is used between consecutive LED transitions, producing a visible running-light effect. In Stage 1, the active LED moves from LED1 to LED8, demonstrating left-to-right operation. In Stage 2, the sequence reverses from LED8 to LED1, demonstrating right-to-left operation. In Stage 3, both directions are continuously repeated to create a back-and-forth running effect. This demonstrates how the custom timer abstraction can provide non-blocking timing for GPIO-based embedded applications.

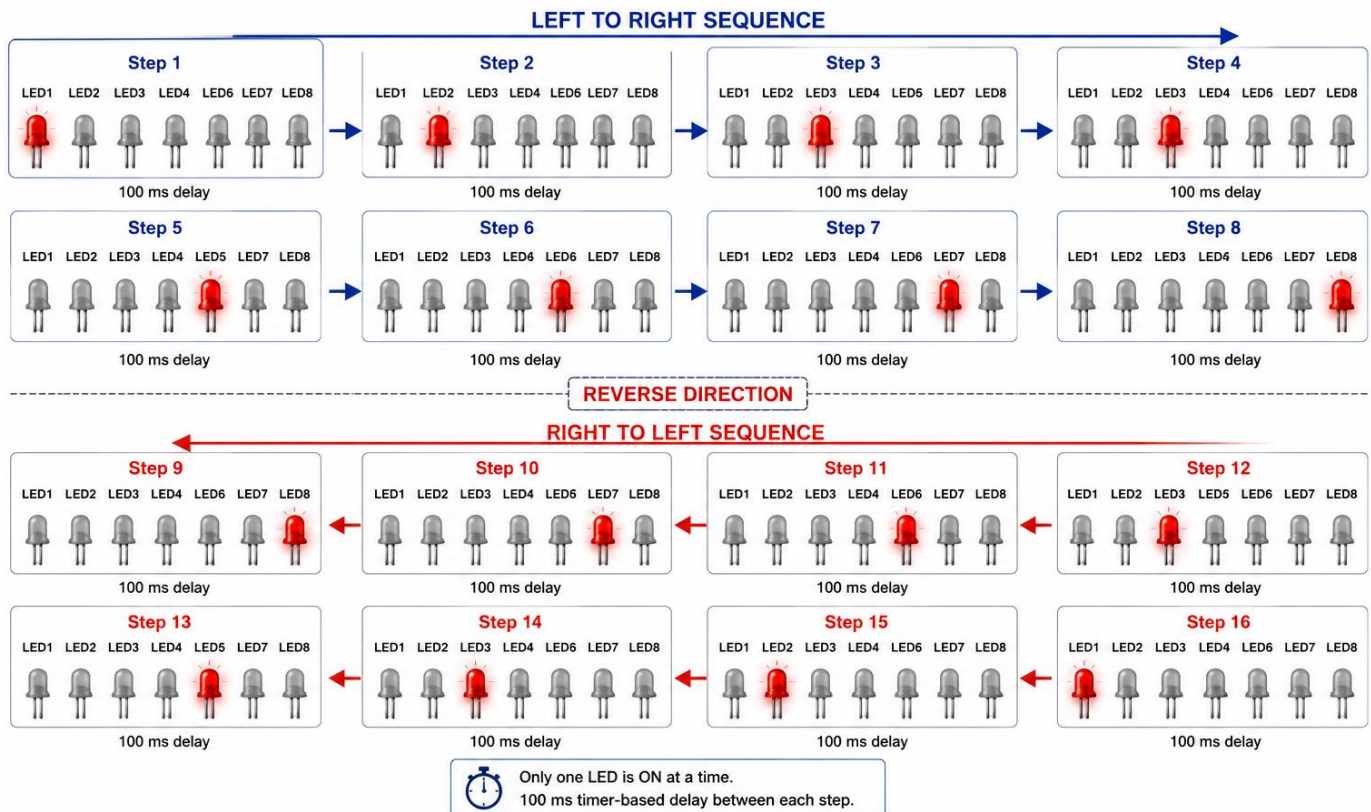
Output Video is upload on Github

Result

- LED1 turns ON while all other LEDs remain OFF.
- LED2 turns ON and LED1 turns OFF, continuing the sequence left to right.
- The same pattern continues until LED8 turns ON.
- The sequence then reverses, with LEDs turning ON from LED8 to LED1.
- A 100 ms delay is maintained between each LED transition.
- The process continuously repeats, producing a smooth left-to-right and right-to-left running LED

STM32F103RB – LED ON/OFF Sequence

Timer-Based LED Running Light (100 ms Delay Between Each Step)



effect.

Conclusion

The experiment successfully demonstrated the use of the STM32F103RB timer and GPIO peripherals for controlling eight LEDs. The timer was configured in interrupt mode to generate periodic timing ticks, which were used to create a precise 100 ms delay between LED transitions. The GPIO pins PA5, PA6, PA7, PB0, PB1, PB2, PB10, and PB11 were successfully used as LED outputs. The LEDs operated in a sequential pattern from left to right and then from right to left. The custom Timer HAL provided functions for starting, stopping, reading ticks, and checking elapsed time. Overall, the experiment improved understanding of STM32 timers, GPIO control, interrupts, software timing, and HAL-based embedded programming.

Github Repository:

<https://github.com/Ahsan-web-hue/STM32F103RB-Timer-Based-8-LED-Running-Light-using-HAL.git>

